



Central university of Haryana

Department of Computer Science & Engineering

(Village Jant-Pali, Mahendergarh, Haryana-123031)

Object Oriented Programming Lab File

Subject Code :- (BT CS 405)



Submitted by :-

Vikas Singh

B.Tech CSE

Roll No:- 24150

Submitted to :-

Mr. Anant R. Bara

(Assistant Professor)

Department of CSE

INDEX

<u>S.No</u>	<u>Program Titles</u>	<u>Page No.</u>
1	WAP to demonstrate inline function for basic arithmetic operations.	2-3
2	WAP to demonstrate default arguments using simple interest calculation.	4-5
3	WAP to demonstrate function overloading for area, sum, and volume calculations.	6-7
4	WAP to demonstrate static data member and static member function.	8-9
5	WAP to demonstrate default, parameterized, copy, and overloaded constructors.	10-12
6	WAP to demonstrate dynamic initialization using constructors.	13-15
7	WAP to demonstrate operator overloading for +, -, ++, ==, and << operators.	16-18
8	WAP to demonstrate single, multiple, multilevel, hierarchical, and hybrid inheritance.	19-22
9	WAP to demonstrate function overriding and runtime polymorphism.	23-24
10	WAP to demonstrate virtual functions and abstract classes using Shape example.	25-27
11	WAP to demonstrate containership using real-life classes (Car–Engine).	28-29
12	WAP to develop a Currency Converter using menu-driven approach.	30-32
13	WAP to design a Bank Management System using OOP concepts.	33-37

Program 1: WAP to demonstrate inline function for basic arithmetic operations.

// Author :- Vikas Singh (241504)

```
#include <iostream>
```

```
using namespace std;
```

```
// Calculator Class to Implement Inline Function
```

```
class Calculator {
```

```
public:
```

```
    inline double add(double a, double b) { return a + b; }
```

```
    inline double subtract(double a, double b) { return a - b; }
```

```
    inline double multiply(double a, double b) { return a * b; }
```

```
    inline double divide(double a, double b) {
```

```
        if (b == 0) {
```

```
            cout << "Error: Division by zero!" << endl;
```

```
            return 0;
```

```
        }
```

```
        return a / b;
```

```
    }
```

```
};
```

```
// Main Function
```

```
int main() {
```

```
    Calculator calc;
```

```

double num1, num2;

cout << "Enter two numbers: ";

cin >> num1 >> num2;


cout << "\n--- Results ---\n";

cout << "Addition:  " << calc.add(num1, num2) << endl;

cout << "Subtraction:  " << calc.subtract(num1, num2) << endl;

cout << "Multiplication: " << calc.multiply(num1, num2) << endl;

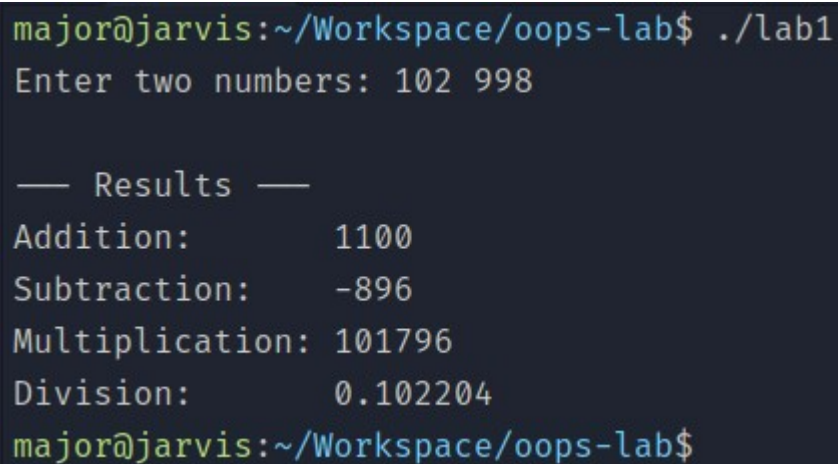
cout << "Division:  " << calc.divide(num1, num2) << endl;


return 0;

}

```

OUTPUT:-



```

major@jarvis:~/Workspace/oops-lab$ ./lab1
Enter two numbers: 102 998

--- Results ---
Addition:      1100
Subtraction:   -896
Multiplication: 101796
Division:      0.102204
major@jarvis:~/Workspace/oops-lab$

```

GitHub Link :-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab1.cpp>

Program 2: WAP to demonstrate default arguments using simple interest code.

// Author :- Vikas Singh (241504)

```
#include <iostream>

using namespace std;

// Simple Intrest Class to Implement Default Arguments

class InterestCalculator {

public:

    double calculateSI(double principal, double rate = 5.0, double time = 1.0) {

        return (principal * rate * time) / 100;

    }

};

// Main Function

int main() {

    InterestCalculator calc;

    double p, r, t;

    cout << "Enter Principal amount: ";

    cin >> p;

    cout << "\n--- Scenario 1: Default Rate (5%) and Time (1yr) ---\n";

    cout << "Interest: " << calc.calculateSI(p) << endl;

    cout << "\nEnter Rate of Interest: ";
```

```

cin >> r;

cout << "--- Scenario 2: Custom Rate, Default Time (1yr) ---\n";

cout << "Interest: " << calc.calculateSI(p, r) << endl;


cout << "\nEnter Time period (years): ";

cin >> t;

cout << "--- Scenario 3: Custom Principal, Rate, and Time ---\n";

cout << "Interest: " << calc.calculateSI(p, r, t) << endl;


return 0;
}

```

OUTPUT :-

```

major@jarvis:~/Workspace/oops-lab$ ./lab2
Enter Principal amount: 2000

— Scenario 1: Default Rate (5%) and Time (1yr) —
Interest: 100

Enter Rate of Interest: 6
— Scenario 2: Custom Rate, Default Time (1yr) —
Interest: 120

Enter Time period (years): 2
— Scenario 3: Custom Principal, Rate, and Time —
Interest: 240

```

GitHub Link :-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab2.cpp>

Program 3: WAP to Demonstrate function overloading for area, sum, and volume calculation.

// Author :- Vikas Singh (241504)

```
#include <iostream>
```

```
using namespace std;
```

```
// Geometry Class to Implement Function Overloading
```

```
class Geometry {
```

```
public:
```

```
    int calculateSum(int a, int b){ return a + b;}
```

```
    double calculateSum(int a, double b){ return a + b;}
```

```
    double calculateSum(double a, int b){ return a + b;}
```

```
    double calculateSum(double a, double b){ return a + b;}
```

```
    double calculateArea(double length){ return length*length;}
```

```
    double calculateArea(double length, double width){ return length * width;}
```

```
    double calculateVolume(double l, double w, double h){ return l * w * h;}
```

```
};
```

```
// Main Function
```

```
int main() {
```

```
    Geometry geo;
```

```
    cout << "Square Area (side 5): " << geo.calculateArea(5.0) << endl;
```

```
    cout << "Rectangle Area (4x6): " << geo.calculateArea(4.0, 6.0) << endl;
```

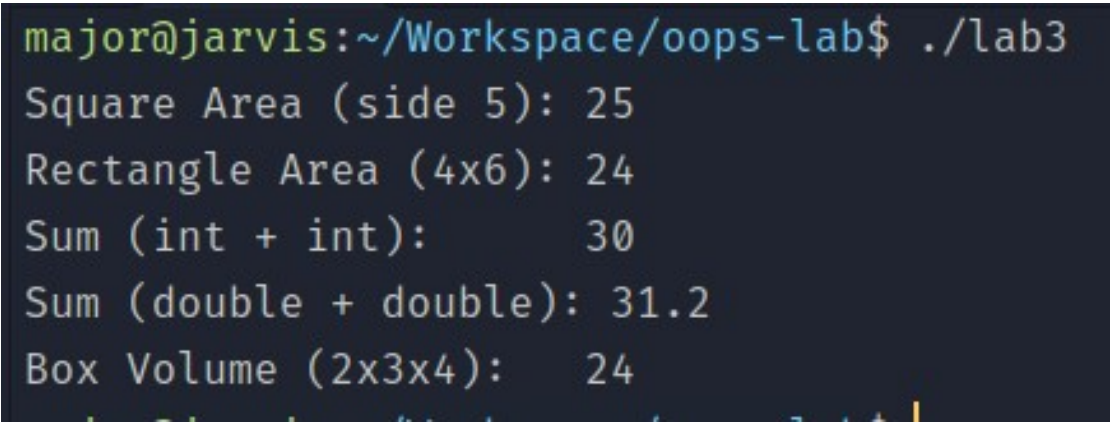
```
cout << "Sum (int + int):   " << geo.calculateSum(10, 20) << endl;

cout << "Sum (double + double): " << geo.calculateSum(10.5, 20.7) << endl;


cout << "Box Volume (2x3x4): " << geo.calculateVolume(2.0, 3.0, 4.0) << endl;


return 0;
}
```

OUTPUT:-

A screenshot of a terminal window with a dark background. The prompt is 'major@jarvis:~/Workspace/oops-lab\$' and the command executed is './lab3'. The output consists of five lines: 'Square Area (side 5): 25', 'Rectangle Area (4x6): 24', 'Sum (int + int): 30', 'Sum (double + double): 31.2', and 'Box Volume (2x3x4): 24'.

```
major@jarvis:~/Workspace/oops-lab$ ./lab3
Square Area (side 5): 25
Rectangle Area (4x6): 24
Sum (int + int): 30
Sum (double + double): 31.2
Box Volume (2x3x4): 24
```

GitHub Link : -

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab3.cpp>

Program 4: WAP to demonstrate static data member & static member function.

// Author :- Vikas Singh (241504)

```
#include <iostream>
using namespace std;

// Counter Class to Implement static data member & static member function
class Counter {
private:
    static int count;
    int id;

public:
    Counter() {
        count++;
        id = count;
    }

    static int getCount() {
        return count;
    }

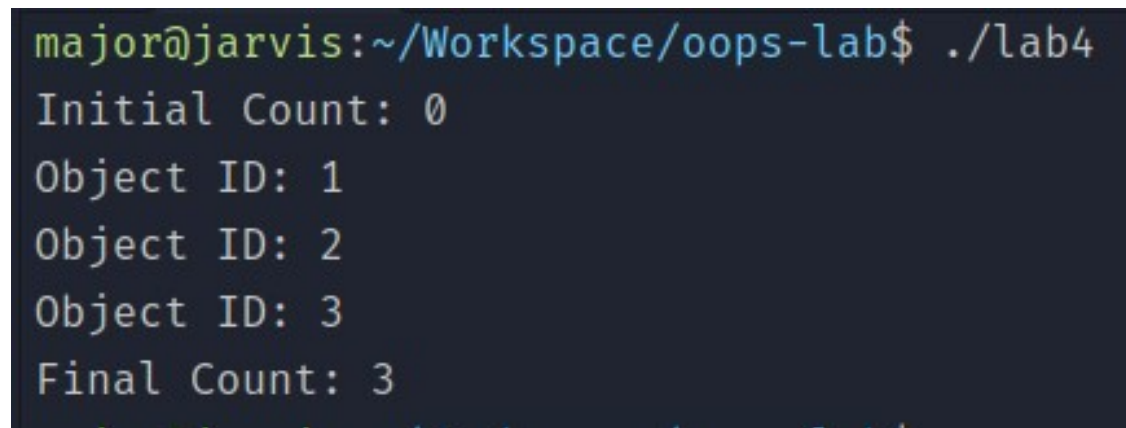
    void displayId() {
        cout << "Object ID: " << id << endl;
    }
};

int Counter::count = 0;

// Main Function
int main() {
    cout << "Initial Count: " << Counter::getCount() << endl;
    Counter c1, c2, c3;
```

```
c1.displayId();  
c2.displayId();  
c3.displayId();  
  
cout << "Final Count: " << Counter::getCount() << endl;  
  
return 0;  
}
```

OUTPUT:-

A terminal window with a dark background and light-colored text. The prompt is 'major@jarvis:~/Workspace/oops-lab\$' and the command is './lab4'. The output consists of five lines: 'Initial Count: 0', 'Object ID: 1', 'Object ID: 2', 'Object ID: 3', and 'Final Count: 3'.

```
major@jarvis:~/Workspace/oops-lab$ ./lab4  
Initial Count: 0  
Object ID: 1  
Object ID: 2  
Object ID: 3  
Final Count: 3
```

GitHub Link : -

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab4.cpp>

Program 5: WAP to demonstrate default, parameterized, copy, and overloaded constructors.

// Author :- Vikas Singh (241504)

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Student Class to Implement default, parameterized, copy, and overloaded constructors
```

```
class Student {
```

```
private:
```

```
    string name;
```

```
    int roll;
```

```
public:
```

```
    Student() {
```

```
        name = "Unknown";
```

```
        roll = 0;
```

```
        cout << "Default Constructor Called" << endl;
```

```
    }
```

```
    Student(string n, int r) {
```

```
        name = n;
```

```
        roll = r;
```

```

        cout << "Parameterized Constructor Called" << endl;
    }

    Student(const Student &s) {
        name = s.name;
        roll = s.roll;
        cout << "Copy Constructor Called" << endl;
    }

    Student(int r) {
        name = "No Name";
        roll = r;
        cout << "Overloaded Constructor (Single Param) Called" << endl;
    }

    void display() {
        cout << "Name: " << name << ", Roll: " << roll << endl << endl;
    }
};

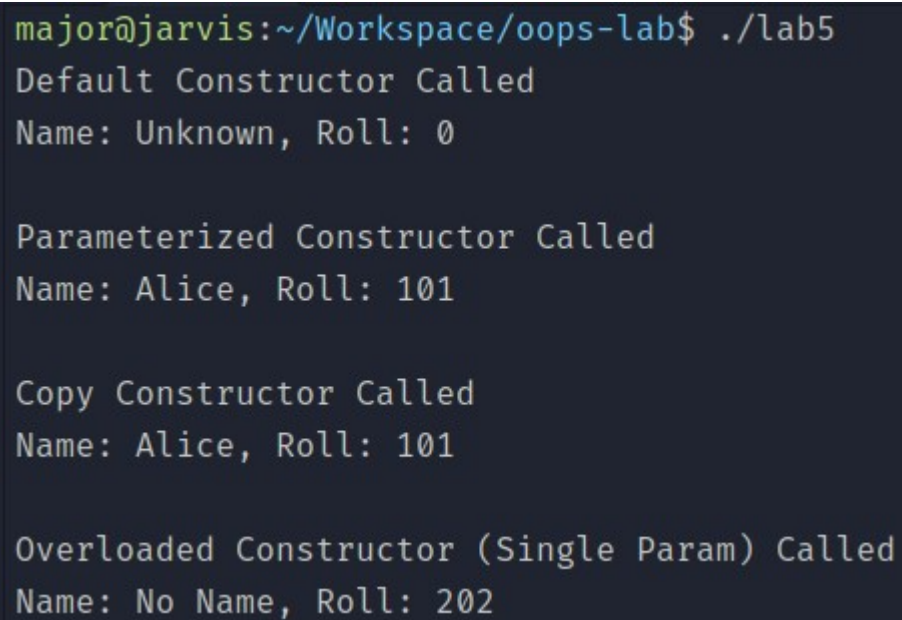
// Main Function

int main() {
    Student s1;
    s1.display();
}

```

```
Student s2("Alice", 101);  
  
s2.display();  
  
Student s3(s2);  
  
s3.display();  
  
Student s4(202);  
  
s4.display();  
  
return 0;  
}
```

OUTPUT:-

A terminal window with a dark background and light green text. The prompt is 'major@jarvis:~/Workspace/oops-lab\$'. The command './lab5' has been executed. The output consists of four lines of text, each preceded by a blank line: 'Default Constructor Called', 'Name: Unknown, Roll: 0'; 'Parameterized Constructor Called', 'Name: Alice, Roll: 101'; 'Copy Constructor Called', 'Name: Alice, Roll: 101'; and 'Overloaded Constructor (Single Param) Called', 'Name: No Name, Roll: 202'.

```
major@jarvis:~/Workspace/oops-lab$ ./lab5  
Default Constructor Called  
Name: Unknown, Roll: 0  
  
Parameterized Constructor Called  
Name: Alice, Roll: 101  
  
Copy Constructor Called  
Name: Alice, Roll: 101  
  
Overloaded Constructor (Single Param) Called  
Name: No Name, Roll: 202
```

GitHub link:

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab5.cpp>

Program 6: WAP to demonstrate dynamic initialization using constructors.

// Author :- Vikas Singh (241504)

```
#include <iostream>

using namespace std;

// FixedDeposit Class to Implement dynamic initialization using constructors

class FixedDeposit {

    long int principal;

    int years;

    float rate;

    float returnValue;

public:

    FixedDeposit() {}

    FixedDeposit(long int p, int y, float r) {

        principal = p;

        years = y;

        rate = r;

        returnValue = principal;

        for (int i = 0; i < y; i++) {

            returnValue = returnValue * (1.0 + r);

        }

    }

}
```

```
}
```

```
FixedDeposit(long int p, int y, int r) {
```

```
    principal = p;
```

```
    years = y;
```

```
    rate = float(r) / 100;
```

```
    returnValue = principal;
```

```
    for (int i = 0; i < y; i++) {
```

```
        returnValue = returnValue * (1.0 + rate);
```

```
    }
```

```
}
```

```
void display() {
```

```
    cout << "Principal: " << principal << endl;
```

```
    cout << "Return Value: " << returnValue << endl << endl;
```

```
}
```

```
};
```

```
// Main Function
```

```
int main() {
```

```
    long int p;
```

```
    int y, R;
```

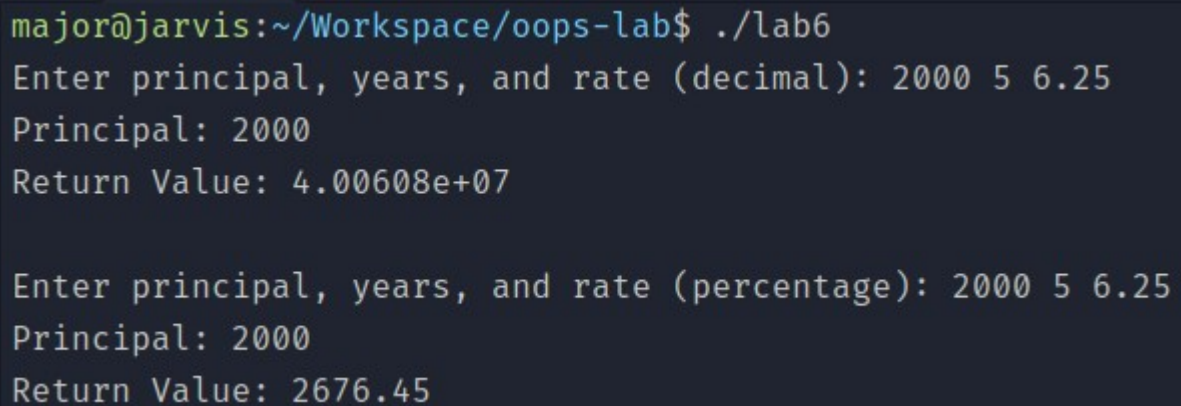
```
    float r;
```

```
    cout << "Enter principal, years, and rate (decimal): ";
```

```
    cin >> p >> y >> r;
```

```
FixedDeposit fd1(p, y, r);  
fd1.display();  
  
cout << "Enter principal, years, and rate (percentage): ";  
cin >> p >> y >> R;  
FixedDeposit fd2(p, y, R);  
fd2.display();  
  
return 0;  
}
```

OUTPUT:-



```
major@jarvis:~/Workspace/oops-lab$ ./lab6  
Enter principal, years, and rate (decimal): 2000 5 6.25  
Principal: 2000  
Return Value: 4.00608e+07  
  
Enter principal, years, and rate (percentage): 2000 5 6.25  
Principal: 2000  
Return Value: 2676.45
```

GitHub Link :-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab6.cpp>

Program 7: WAP to demonstrate operator overloading for +, -, ++, ==, and << operators.

// Author :- Vikas Singh (241504)

```
#include <iostream>

using namespace std;

// Point Class to Implement operator overloading

class Point {
private:
    int x, y;

public:
    Point(int x = 0, int y = 0) : x(x), y(y) {}

    Point operator+(const Point& other) {
        return Point(x + other.x, y + other.y);
    }

    Point operator-(const Point& other) {
        return Point(x - other.x, y - other.y);
    }

    Point& operator++() {
        x++;
```

```

        y++;

        return *this;
    }

    bool operator==(const Point& other) {
        return (x == other.x && y == other.y);
    }

    friend ostream& operator<<(ostream& os, const Point& p) {
        os << "(" << p.x << ", " << p.y << ")";
        return os;
    }
};

// Main Function
int main() {
    Point p1(10, 20), p2(5, 5);

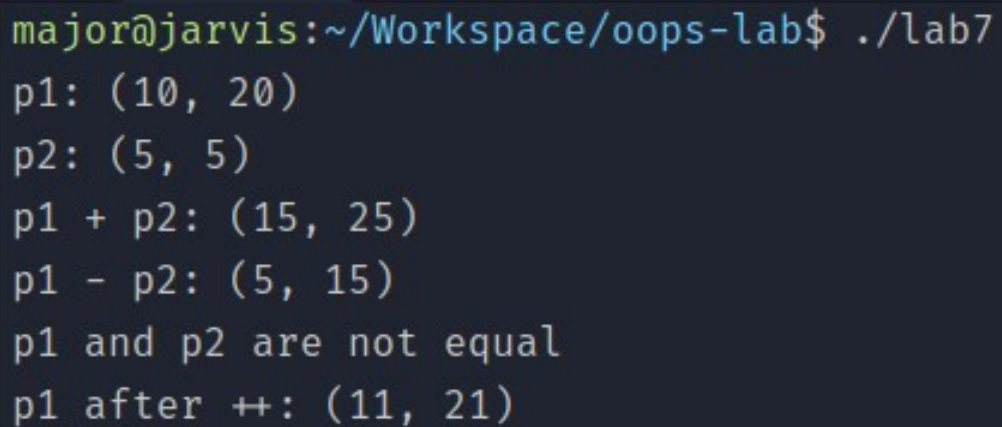
    Point p3 = p1 + p2;
    Point p4 = p1 - p2;

    cout << "p1: " << p1 << endl;
    cout << "p2: " << p2 << endl;
    cout << "p1 + p2: " << p3 << endl;
    cout << "p1 - p2: " << p4 << endl;

```

```
if (p1 == p2) {  
    cout << "p1 and p2 are equal" << endl;  
} else {  
    cout << "p1 and p2 are not equal" << endl;  
}  
  
++p1;  
cout << "p1 after ++: " << p1 << endl;  
  
return 0;  
}
```

OUTPUT:-

A terminal window with a dark background and light-colored text. The prompt is 'major@jarvis:~/Workspace/oops-lab\$' and the command is './lab7'. The output consists of several lines: 'p1: (10, 20)', 'p2: (5, 5)', 'p1 + p2: (15, 25)', 'p1 - p2: (5, 15)', 'p1 and p2 are not equal', and 'p1 after ++: (11, 21)'.

```
major@jarvis:~/Workspace/oops-lab$ ./lab7  
p1: (10, 20)  
p2: (5, 5)  
p1 + p2: (15, 25)  
p1 - p2: (5, 15)  
p1 and p2 are not equal  
p1 after ++: (11, 21)
```

GitHub Link: -

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab7.cpp>

Program 8: WAP to demonstrate single, multiple, multilevel, hierarchical and hybrid inheritance.

// Author :- Vikas Singh (241504)

```
#include <iostream>

#include <string>

using namespace std;

// Base class for Single, Multilevel, Hierarchical, and Hybrid

class Animal {

public:

    void eat() { cout << "Eating..." << endl; }

};

// SINGLE INHERITANCE

class Dog : public Animal {

public:

    void bark() { cout << "Barking..." << endl; }

};

// MULTILEVEL INHERITANCE

class Puppy : public Dog {

public:

    void weep() { cout << "Weeping..." << endl; }

};
```

```

// MULTIPLE INHERITANCE

class Mammal {

public:

    void breathe() { cout << "Breathing..." << endl; }

};

class Bat : public Animal, public Mammal {

public:

    void fly() { cout << "Flying..." << endl; }

};

// HIERARCHICAL INHERITANCE

class Cat : public Animal {

public:

    void meow() { cout << "Meowing..." << endl; }

};

// HYBRID INHERITANCE

class HybridPet : public Dog, public Cat {

public:

    void show() { cout << "I am a hybrid pet." << endl; }

};

// Main Function

int main() {

    cout << "--- Single Inheritance ---" << endl;

```

```
Dog d;  
  
d.eat();  
  
d.bark();  
  
  
cout << "\n--- Multilevel Inheritance ---" << endl;  
  
Puppy p;  
  
p.eat();  
  
p.bark();  
  
p.weep();  
  
  
cout << "\n--- Multiple Inheritance ---" << endl;  
  
Bat b;  
  
b.eat();  
  
b.breathe();  
  
b.fly();  
  
  
cout << "\n--- Hierarchical Inheritance ---" << endl;  
  
Cat c;  
  
c.eat();  
  
c.meow();  
  
  
cout << "\n--- Hybrid Inheritance ---" << endl;  
  
HybridPet hp;  
  
hp.bark();  
  
hp.meow();
```

```
hp.show();

return 0;
}
```

OUTPUT:-

```
major@jarvis:~/Workspace/oops-lab$ ./lab8
— Single Inheritance —
Eating ...
Barking ...

— Multilevel Inheritance —
Eating ...
Barking ...
Weeping ...

— Multiple Inheritance —
Eating ...
Breathing ...
Flying ...

— Hierarchical Inheritance —
Eating ...
Meowing ...

— Hybrid Inheritance —
Barking ...
Meowing ...
I am a hybrid pet.
```

GitHub Link:-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab8.cpp>

Program 9: WAP to demonstrate function overriding & runtime polymorphism.

// Author :- Vikas Singh (241504)

```
#include <iostream>
```

```
using namespace std;
```

```
// Base class for virtual functions
```

```
class Base {
```

```
public:
```

```
    virtual void display() { cout << "Display from Base class \n"; }
```

```
    virtual void show() { cout << "Show from Base class \n" ; }
```

```
};
```

```
// Derived class to implement runtime overriding and polymorphism
```

```
class Derived : public Base {
```

```
public:
```

```
    void display() override { cout << "Display from Derived class (Overridden) \n"; }
```

```
    void show() override { cout << "Show from Derived class (Overridden) \n" ; }
```

```
};
```

```
// Main Function
```

```
int main() {
```

```
    Base* ptr;
```

```
    Base baseObj;
```

```
    Derived derObj;
```



```
ptr = &baseObj;

ptr->display();

ptr->show();


cout << "-----" << endl;

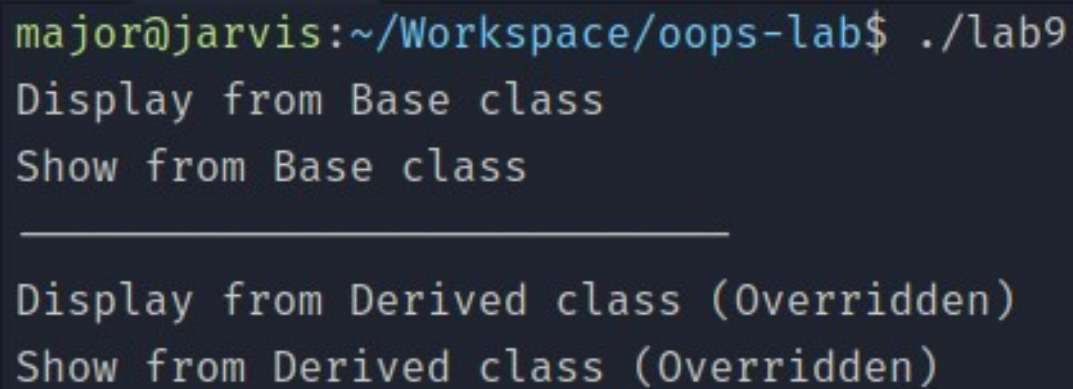
ptr = &derObj;

ptr->display();

ptr->show();


return 0;
}
```

OUTPUT:-

A terminal window with a dark background. The prompt is 'major@jarvis:~/Workspace/oops-lab\$'. The command './lab9' has been executed. The output is: 'Display from Base class', 'Show from Base class', followed by a horizontal line, then 'Display from Derived class (Overridden)', and 'Show from Derived class (Overridden)'.

```
major@jarvis:~/Workspace/oops-lab$ ./lab9
Display from Base class
Show from Base class
-----
Display from Derived class (Overridden)
Show from Derived class (Overridden)
```

GitHub Link:-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab9.cpp>

Program 10: WAP to demonstrate function virtual functions and abstract classes using shape example.

// Author :- Vikas Singh (241504)

```
#include <iostream>

using namespace std;

// Base Class

class Shape {
protected:
    double width, height;

public:
    void setDimensions(double w, double h) {
        width = w;
        height = h;
    }
    virtual void calculateArea() = 0;
};

// Abstract class

class Rectangle : public Shape {
public:
    void calculateArea() override {
        cout << "Area of Rectangle: " << width * height << endl;
```

```

    }

};

// Abstract class

class Triangle : public Shape {

public:

    void calculateArea() override {

        cout << "Area of Triangle: " << 0.5 * width * height << endl;

    }

};

// Main Function

int main() {

    Shape* ptr;

    Rectangle rect;

    Triangle tri;

    ptr = &rect;

    ptr->setDimensions(10, 5);

    ptr->calculateArea();

    ptr = &tri;

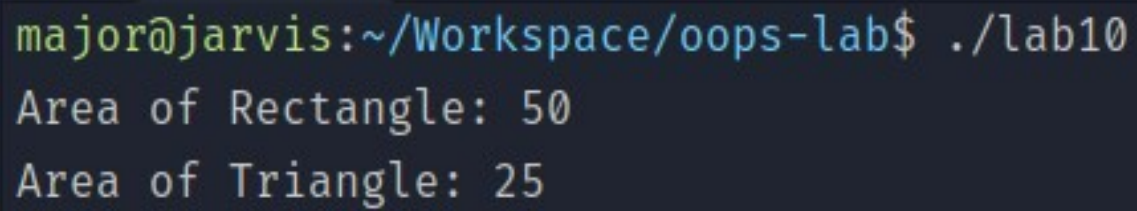
    ptr->setDimensions(10, 5);

    ptr->calculateArea();

```

```
    return 0;  
}
```

OUTPUT:-

A terminal window with a dark background. The prompt is 'major@jarvis:~/Workspace/oops-lab\$'. The command './lab10' has been executed, resulting in two lines of output: 'Area of Rectangle: 50' and 'Area of Triangle: 25'.

```
major@jarvis:~/Workspace/oops-lab$ ./lab10  
Area of Rectangle: 50  
Area of Triangle: 25
```

GitHub Link:-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab10.cpp>

Program 11: WAP to demonstrate containership using real-life classes.

// Author :- Vikas Singh (241504)

```
#include <iostream>

#include <string>

using namespace std;
```

```
// Contained Class
```

```
class Engine {

private:

    int horsepower;

    string type;

public:

    Engine(int hp, string t) : horsepower(hp), type(t) {}

    void showEngineDetails() {

        cout << "Engine: " << type << " (" << horsepower << " HP)" << endl;

    }

};
```

```
// Container Class
```

```
class Car {

private:

    string model;

    Engine eng;
```

```
public:

    Car(string m, int hp, string t) : model(m), eng(hp, t) {}

    void showCarDetails() {

        cout << "Car Model: " << model << endl;

        eng.showEngineDetails();

    }

};

// Main function

int main() {

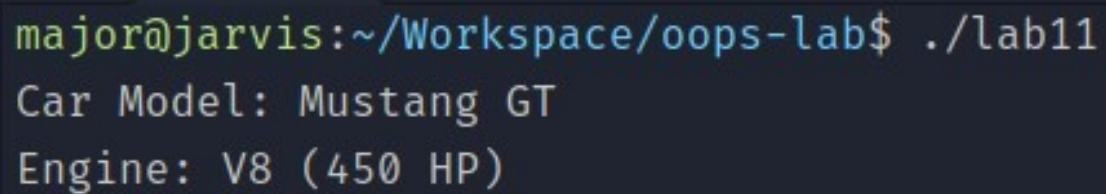
    Car myCar("Mustang GT", 450, "V8");

    myCar.showCarDetails();

    return 0;

}
```

OUTPUT:-



```
major@jarvis:~/Workspace/oops-lab$ ./lab11
Car Model: Mustang GT
Engine: V8 (450 HP)
```

GitHub Link:-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab11.cpp>

Program 12: WAP to develop a Currency Converter using menu-driven approach.

// Author :- Vikas Singh (241504)

```
#include <iostream>

#include <iomanip>

using namespace std;

// Currency Converter Class

class CurrencyConverter {

public:

    void usdToInr(double amount) {

        cout << fixed << setprecision(2);

        cout << amount << " USD = " << (amount * 83.50) << " INR" << endl;

    }

    void inrToUsd(double amount) {

        cout << fixed << setprecision(2);

        cout << amount << " INR = " << (amount / 83.50) << " USD" << endl;

    }

    void usdToEur(double amount) {

        cout << fixed << setprecision(2);

        cout << amount << " USD = " << (amount * 0.92) << " EUR" << endl;

    }

}
```

```

void eurToUsd(double amount) {
    cout << fixed << setprecision(2);

    cout << amount << " EUR = " << (amount / 0.92) << " USD" << endl;
}
};

```

```
// Main Function
```

```

int main() {
    CurrencyConverter cc;

    int choice;

    double amount;

    while (true) {
        cout << "\n--- CURRENCY CONVERTER MENU ---" << endl;

        cout << "1. USD to INR" << endl;

        cout << "2. INR to USD" << endl;

        cout << "3. USD to EUR" << endl;

        cout << "4. EUR to USD" << endl;

        cout << "5. Exit" << endl;

        cout << "Enter your choice: ";

        cin >> choice;

        if (choice == 5) {
            cout << "Exiting program..." << endl;

            break;
        }
    }
}

```



```

    if (choice >= 1 && choice <= 4) {

        cout << "Enter amount: ";

        cin >> amount;

    }

    switch (choice) {

        case 1: cc.usdToInr(amount); break;

        case 2: cc.inrToUsd(amount); break;

        case 3: cc.usdToEur(amount); break;

        case 4: cc.eurToUsd(amount); break;

        default: cout << "Invalid choice! Please try again." << endl;

    }

}

return 0;

}

```

OUTPUT:-

```
major@jarvis:~/Workspace/oops-lab$ ./lab12
```

```
— CURRENCY CONVERTER MENU —
```

```
1. USD to INR
```

```
2. INR to USD
```

```
3. USD to EUR
```

```
4. EUR to USD
```

```
5. Exit
```

```
Enter your choice: 1
```

```
Enter amount: 12
```

```
12.00 USD = 1002.00 INR
```

```
major@jarvis:~/Workspace/oops-lab$ ./lab12
```

```
— CURRENCY CONVERTER MENU —
```

```
1. USD to INR
```

```
2. INR to USD
```

```
3. USD to EUR
```

```
4. EUR to USD
```

```
5. Exit
```

```
Enter your choice: 2
```

```
Enter amount: 2000
```

```
2000.00 INR = 23.95 USD
```

GitHub Link:-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab12.cpp>

Program 13: WAP to design a Bank Management System using OOP concepts.

// Author :- Vikas Singh (241504)

```
#include <iostream>

#include <string>

#include <iomanip>

using namespace std;

// Banc Account Class

class BankAccount {

private:

    string accountHolder;

    int accountNumber;

    double balance;

public:

    BankAccount(string name, int accNum, double initialDeposit) {

        accountHolder = name;

        accountNumber = accNum;

        balance = initialDeposit;

        cout << "\nAccount created successfully for " << accountHolder << endl;

    }

}
```

```
void deposit(double amount) {  
    if (amount > 0) {  
        balance += amount;  
        cout << "Successfully deposited: $" << amount << endl;  
    } else {  
        cout << "Invalid deposit amount!" << endl;  
    }  
}
```

```
void withdraw(double amount) {  
    if (amount > balance) {  
        cout << "Insufficient balance! Current balance: $" << balance << endl;  
    }  
    else if (amount <= 0) {  
        cout << "Invalid withdrawal amount!" << endl;  
    }  
    else {  
        balance -= amount;  
        cout << "Successfully withdrawn: $" << amount << endl;  
    }  
}
```

```
void displayBalance() {  
    cout << fixed << setprecision(2);  
    cout << "\n--- Account Details ---" << endl;
```

```

        cout << "Account Holder: " << accountHolder << endl;

        cout << "Account Number: " << accountNumber << endl;

        cout << "Current Balance: $" << balance << endl;

    }

};

// Main Function

int main() {

    string name;

    int accNum;

    double amount;

    int choice;

    cout << "--- Welcome to the Bank Management System ---" << endl;

    cout << "Enter Account Holder Name: ";

    getline(cin, name);

    cout << "Enter Account Number: ";

    cin >> accNum;

    cout << "Enter Initial Deposit: ";

    cin >> amount;

    BankAccount myAccount(name, accNum, amount);

```

```
do {

    cout << "\n1. Deposit Money" << endl;

    cout << "2. Withdraw Money" << endl;

    cout << "3. Check Balance" << endl;

    cout << "4. Exit" << endl;

    cout << "Enter your choice: ";

    cin >> choice;

    switch (choice) {

        case 1:

            cout << "Enter amount to deposit: ";

            cin >> amount;

            myAccount.deposit(amount);

            break;

        case 2:

            cout << "Enter amount to withdraw: ";

            cin >> amount;

            myAccount.withdraw(amount);

            break;

        case 3:

            myAccount.displayBalance();

            break;

        case 4:

            cout << "Thank you for using our banking services!" << endl;
```

```

        break;

    default:

        cout << "Invalid choice! Please try again." << endl;

    }

} while (choice != 4);

return 0;

}

```

OUTPUT:-

```

major@jarvis:~/Workspace/oops-lab$ ./lab13
— Welcome to the Bank Management System —
Enter Account Holder Name: Vikas Singh
Enter Account Number: 0897
Enter Initial Deposit: 2000

Account created successfully for Vikas Singh

```

```

1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Exit
Enter your choice: 1
Enter amount to deposit: 2000
Successfully deposited: $2000

```

```

1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Exit
Enter your choice: 2
Enter amount to withdraw: 100
Successfully withdrawn: $100

```

GitHub Link:-

<https://github.com/vikasingh0897/OOPs-Lab-Repo/blob/main/lab13.cpp>